Applied Quantum Algorithms

Mini Project: Expressivity of the generating set (T.2.E)

Luigi Cippone

November 10, 2026

Abstract

In this project the ability of a Neural Network and a Parameterized quantum circuit to learn a synthetic dataset is tested for different input encodings. Three different datasets are produced and three different encodings are tested for the PQC. It i shown that the NN outperforms the PQC for a linearly seperated dataset while for more complex datasets the models are mostly equivalent (with PQC) having a sligth edge. The results are inconclusive but it seems like more complex encodings are required for a PQC to rightly outperform NNs.

1 Introduction

This project is mainly considered with the encoding of data into a quantum circuit and its impact on a learning agent's ability (classical or quantum) to learn said data. Specifically the aim is to understand how different encoding schemes and encoding layers change the ability of parameterized quantum circuits and classical neural networks to learn. Because achieving the maximum accuracy on some real-world dataset is not the main focus of this project the dataset is going to be artificially generated (by what is going to be called a generator) in order to restrict the learners hypothesis class and then this dataset is going to be learnt (by a classifier). This is achieved by using classical neural networks (NNs) and parameterized quantum circuits (PQC) as generators and classifiers.
In particular, the considered PQC is a low-depth variational quantum circuit, which has the potential to run on near-term quantum devices given their good adaptability to error correction. To begin with, in order to generate the data with a PQC with n qubits, the input vectors are encoded in the 2^n amplitudes by means of a quantum circuit.

One would expect that all things considered equal the PQC's ability to learn would be greater than that of a classical neural network due to the fact that -as stated above- Hilbert space is exponentially bigger than classical state space. To do this 3 artificial datasets are generated by classifying 2d points using a PQC. One dataset linearly separable and two with increased complexity. Afterwards this data is fed to a classical NN and another PQC architecture and

their performance is compared. It is shown that the best classifier for the linearly generated dataset is the NN by a landslide. However, for the same depth the PQC sometimes scores higher than the NN for the more complex datasets. The results are not completely robust, but still one can make qualitative assumptions for the learning abillity of the networks. We use an explicit variational method for classification of the dataset where the circuits weights are updated at every iteration.

2 PQC and artificial data generation

A static state preparation circuit S must be applied to the initial ground state in order to prepare such a quantum state. The data is non-linearly mapped to the feature space where a hyperplane is built to divide the labeled samples.

2.1 Quantum Circuit

A PQC can be defined by the mapping:

$$\rho(0) \mapsto \rho(t; \Theta) \tag{1}$$

where the initial state ρ is mapped onto a final state $\rho(\Theta)$ which is characterised by a set of parameters Θ. These parameters can be seperated into two categories namely fixed and tunable parameters x and θ. Of course in a real world quantum computers one should take into account the specific computers properties [1]. The fixed parameters are going to be responsible for the input encoding while the free parameters are going to be optimised during training.

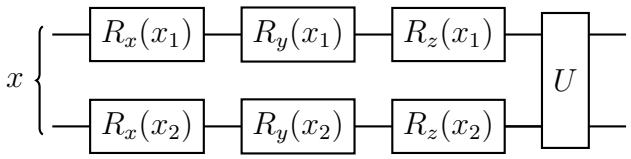
2.2 Data Generation

The synthetic dataset is generated in a manner similar to [1804.11326]. The data are being generated by creating a set of random points in $[0, 2\pi]^2$. Then this set is fed into a PQC of 2 qubits that rotates each qubit i by a set amount x_i in each direction. A representation of the circuit is shown in 1. The data are being generated by repeatedly running the cirquit and measureing the following expected value

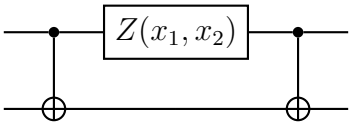
$$\langle C_2 Z(x_1, x_2) C_2 \rangle \tag{2}$$

where C_2 is a CNOT gate with qubit 2 as its target. A schematic representation is show in fig. 1b. $Z(x_1, x_2)$ rotates the first qubit by an angle $f(x_1, x_2)$ in the z-direction. The specific form of this function changes the shape of the classification. If we omit either x_1 or x_2 dependence we produce a linearly seperated dataset as the classification is only performed on a line, however different combinations of x_1 and x_2 produce more interesting datasets as can be shown in fig.2. Afterwards the dataset is resampled to achieve balanced data so that both classes are represented equally.

[1] One such property can be its computers physical geometry as it impacts the way local operations are permitted to act on the system. However since this theoretical approach is hardware independent all parameters not regarding input encoding or trainning are to be omitted.



(a) Encoding circuit for data generation



(b) Schematic showing the operator for which the expected value is measured

Figure 1: Schematic representation of the circuit used to generate the artificial dataset. The input x is encoded into two qubits by rotating each by the same amount in all 3 directions

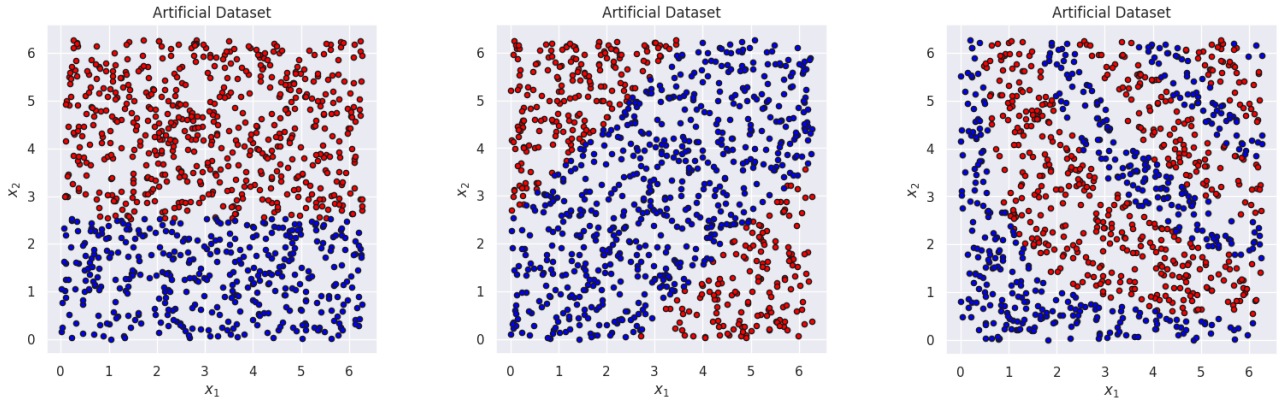


Figure 2: Resulting datasets for 3 different choices of the output rotation function $f(x_1, x_2)$. From left to right: First image shows the resulting dataset of $f = x_2$ (dataset 1) producing a linearly separated dataset, second image corresponds to the case where $f = x_1 - x_2$ (dataset 2) and two descision boundaries need be drawn. Lastly $f = x_1 x_2$ (dataset 3) produces this even more complex wavy dataset.

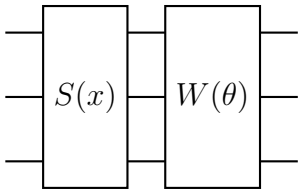


Figure 3: A simplistic version of the parametrised cirquit used for classification. $S(x)$ is the encoding part of the circuit and $W(\theta)$ is the variational part.

3 Training

For training on the dataset we use both a PQC based model and a classical Neural Network. The PQC is comprised of and encoding and a variational part (shown in fig.3). The ability of these two as a classifier is going to be tested with the dataset generated by the generator cirquit. Both of them will have 3 inputs (3 qubits for the PQC and 3 input neurons for the NN) as we want to encode both x_1 and x_2 as well as a combination of x_1 and x_2 in the PQC as a kernel method to make the classification easier.

3.1 PQC

We create a PQC architecture architecture (like the one shown in figure 4) We keep the type of the variational layers fixed and only vary their number. A square loss function is used for optimization and the loss function is optimized using the "COBYLA" method as the derivatives of the loss function cannot be accesed by a classical optimizer. Three different encodings will be tested:

$$\begin{aligned}
&\text{Encoding 1} : S(x) = R_z(x) \\
&\text{Encoding 2} : S(x) = R_z(x)R_y(x) \\
&\text{Encoding 3} : S(x) = R_z(x)R_y(x)R_z(x)
\end{aligned}$$

The number of layers in the variational circuit is also going to be varied but no more than 5 layers are going to be used (15 in just one extreme case).

3.2 Neural Network

The classical classifier is a fully connected feed-forward neural network with 3 input neurons and l layers. The loss function used is 'binary crossentropy' and 'adam' as optimizer. The model runs for a maximum of 80 epochs. It should be noted that the amount of parameters for an l-layer neural network is $2q + q^2l$ where q is the number of qubits of the system while an l-layer PQC will have ql parameters due to the structure of the variational circuit and the NN (fully connected feed forward with single outpus vs 3 rotations + CNOT per layer).

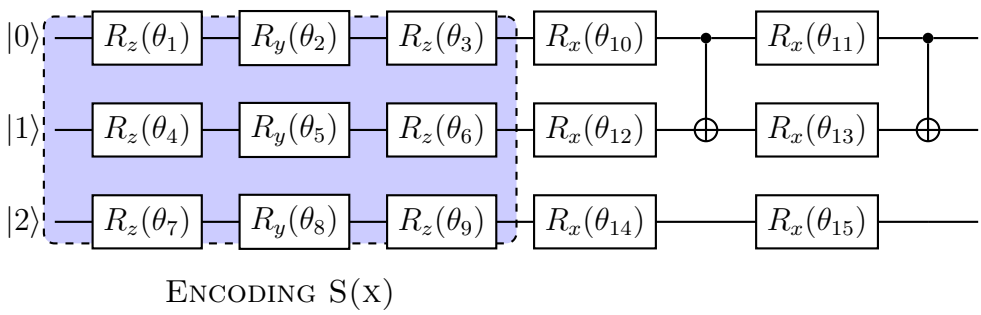


Figure 4: Representation of one of the circuits used for learning the dataset. The dashed line encloses the encoding part of the circuit which is comprised by x,y,z rotations which are proportional to the dataset x. The following layers are the variational circuit which again is comprised of x rotations and a single CNOT gate between the two qubits. Each sequence of rotations-CNOT is considered as 1 variational layer (here depicted 2). In this sense parameters $\theta_1 - \theta_8$ depend on the training set while parameters $\theta_9 - \theta_{14}$ are to be optimized during learning.

4 Results

Here the results are presented for different encodings and classifier layers for the 3 different datasets. We use 3 qubits for the PQC, qubits 1 and 2 are to be encoded by rotations proportional to x_1 and x_2 and qubit 3 is going to be encoded using a rotation that is a function of x_1 and x_2 we use this last qubit to bias the classifier towards a specific shape, however the reasoning of using such an (arbitrary) recasting of the data is not very well motivated but still helps the PQC learn better.

For the NN we use 3 neurons for each layer (to be as equivalent as possible to the PQC) which produces a very truncated network compared to modern architectures.

4.1 Dataset 1

The first dataset being linearly seperated should be the one we achieve the highest accuracy. Running on a Neural Network, even for 1 layer a reasonably good accuracy is achieved as can be seen in figure 6. However the PQC performs classifications almost at random.

Even for a deep variational circuit the classifier does not seem do be able to make out the pattern in the data. This can be seen in the comparison of figure 5 where we used 15 layers for the variational circuit with the most complex "Encoding 3".

Intermediate cases (with few layers and or 'Encoding 2') are ommited here as the first dataset was not suited to be classified by a quantum computer to begin with. The fact that the data are linearly seperated means that there is no reason for augmentation in a greater dimension through kernel methods for the classifier to be able to draw a linear boundary. We conclude Classical Neural networks are better suitted for datasets of this form.

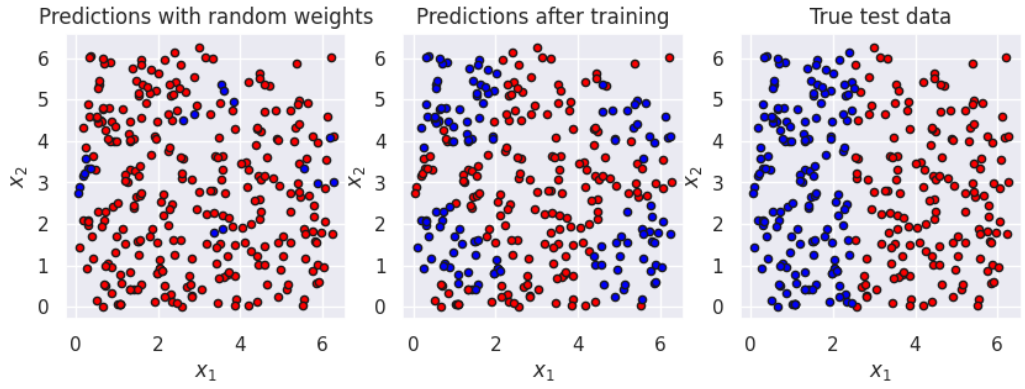


Figure 5: Comparison of the PQC classification and the actual data. In the extreme case of 15 layers and 45 trainable parameters with 'Encoding 3' the classifier only achieves an accuracy of 0.64 on the training set and 0.61 on the validation set. One can see that the resulting pattern cannot be attributed to the original pattern of the data.

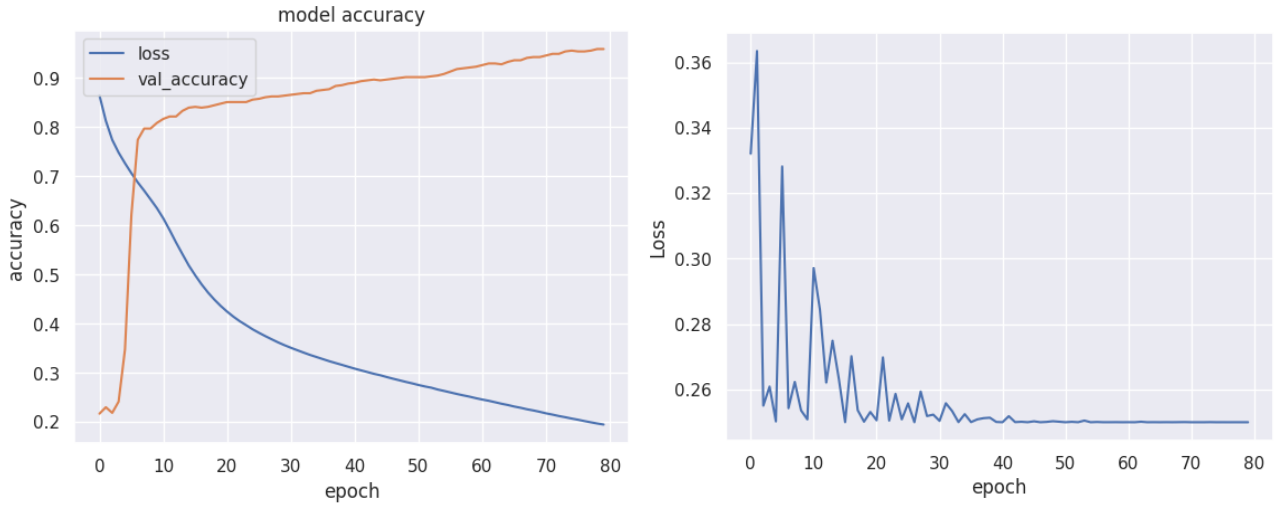


Figure 6: Loss function of a Neural Network (left) on the first dataset with 1 layer and 13 trainable parameters. The model achieves after 80 epochs a 0.959 accuracy and a 0.917 validation accuracy. Loss function of a PQC (right) on the first dataset with 5 layers and 15 trainable parameters. The model after 80 epochs achieves a 0.5 accuracy

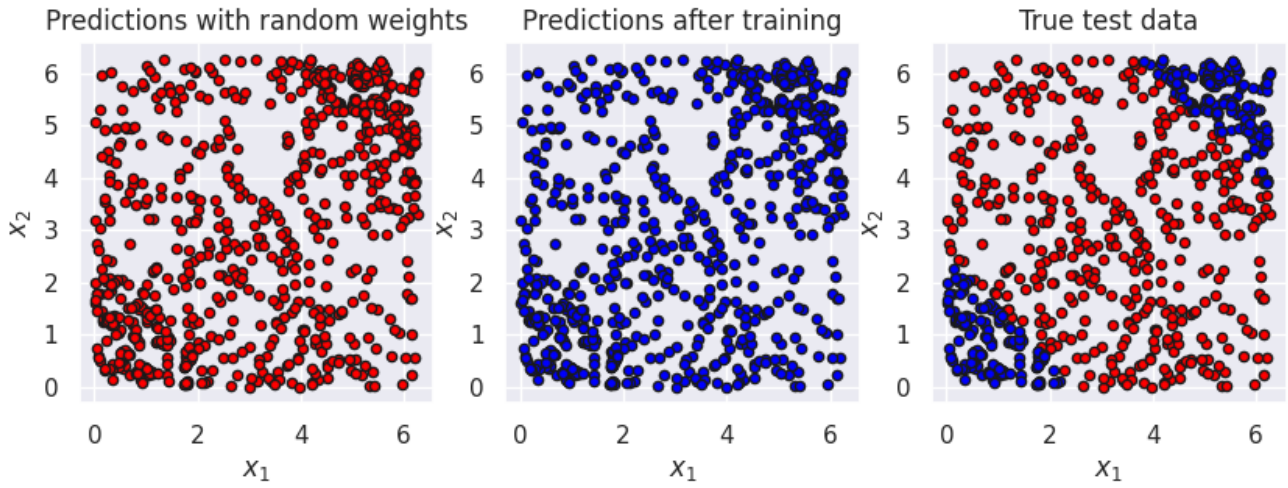


Figure 7: True and trained labels of the dataset 2 for the PQC. One can see that the classification is done by assigning a single layer.

4.2 Dataset 2

For the second dataset the situation starts becoming more interesting. The PQC generally outperforms a shallow neural network. For encodings 2 and 3. The neural networks accuracy for this dataset when one layer is used hovers around 0.7 which is not unusual given the small width of the network.

4.2.1 Encoding 1:

For the first encoding the PQCs classification seems random. It achieves close to 50% accuracy and it does so by assigning the same label to the whole dataset as can be seen in figure. It is not clear to me why this behaviour is exhibited especially with a balanced dataset.7

4.2.2 Encoding 2:

Beggining with the simple case of 1 layer NN and PQC it is found that the PQC performs better than the NN when 'Encoding 2' is used 8. With a $\approx 5\%$ better accuracy on the training set and a $\approx 7\%$ better accuray on the validation set.

4.2.3 Encoding 3:

Encoding 3 gives the best performance on the dataset 2. For the shallow regime of only 1 layer the PQC achieves an accuracy of 0.849 and a validation accuracy of 0.812 (a $10+\%$ increase compared to the NN). Comparison of the true and classified datasets can be found on figure 9

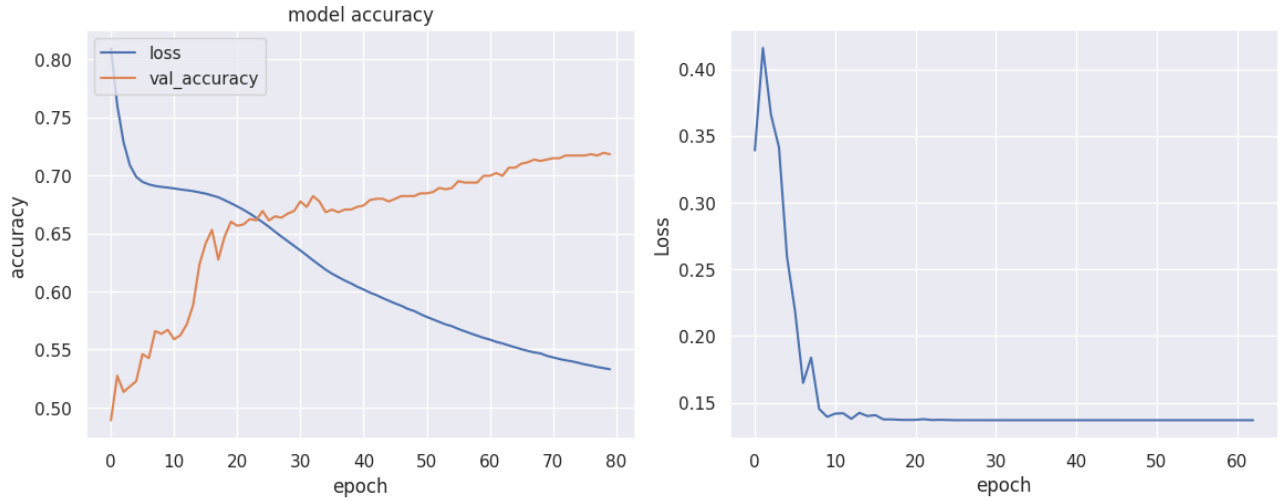


Figure 8: Loss function of a NN (left) and PQC(right). Both comprised of 1 layer (13 parameters for the NN and 3 for the PQC) with 'Encoding 2'. The NN achieves an accuracy of 0.733 and a validation accuracy of 0.718. On the flipside the PQC slightly outperforms the NN even with 1/4 of the parameters achieving an accuracy of 0.789 on the training set and 0.783 on the validation set.

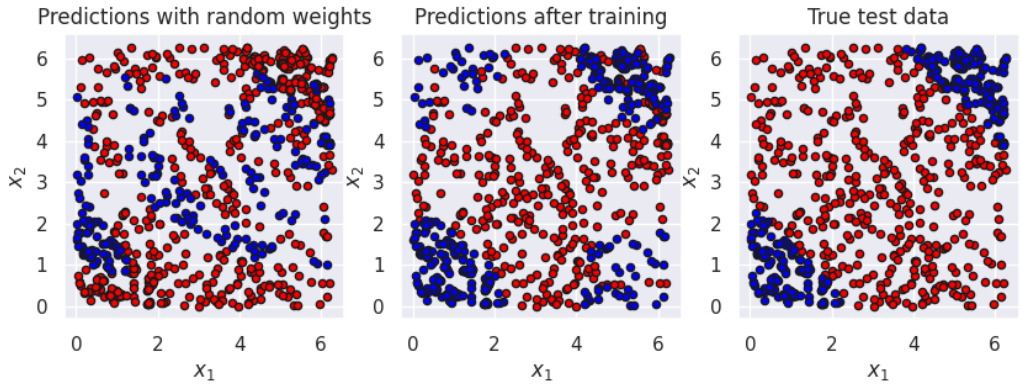


Figure 9: Comparison of the initial, final predictions on the second dataset with 'Encoding 3'. The PQC achieves an accuracy of 0.849 and a validation accuracy of 0.812

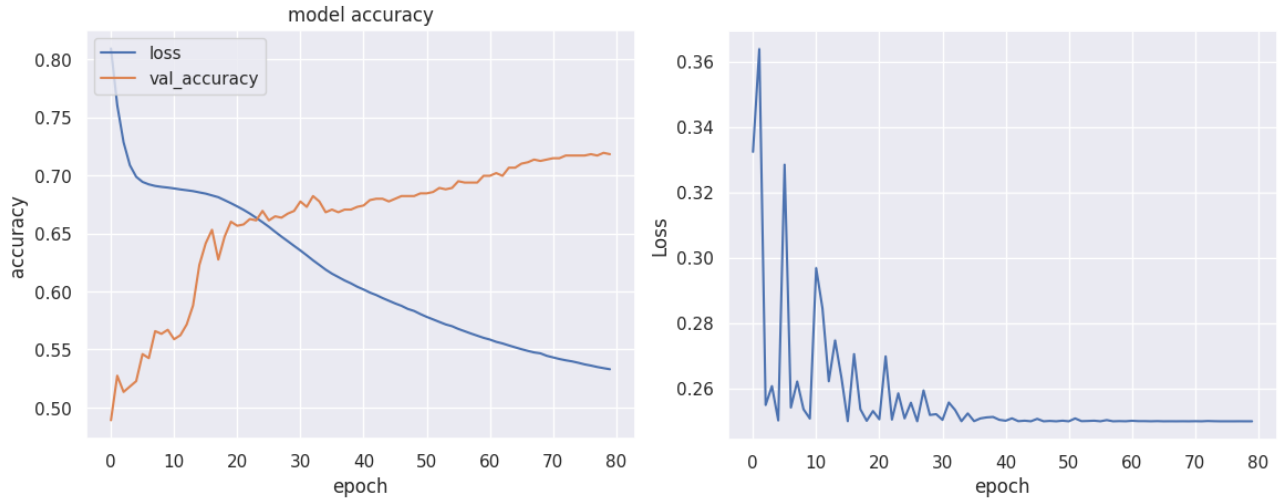


Figure 10: NN loss and accuracy (left) with 5 layers and 61 trainable parameters achieving an accuracy of 0.835 on the training set and 0.823 on the test set. PQC loss(right) with 'Encoding 1' the same number of layers and 15 parameters (note PQC loss is identical to the one in 6). This goes to show that with 'Encoding 1' deep architectures for the PQC while able to minimize the loss function do not provide better accuracy.

4.2.4 Deep Networks Regime

Up until now we considered shallow regimes and as expected NNs generally outperform PQC in the first dataset and the other way aroun in the second dataset. One might be inclined to think that the PQC ability to achieve better accuracy than the NN is due to the fact that we are using a simple NN with only one layer. In the following we run both the NN and the PQC for 5 layers (so 'deep' is to be taken with a grain of salt here) and for the 3 different encodings.

The results for 'Encoding 1' can be seen in figure 10. It should be noted that despite both networks having the same depth there is a vast discrepence in the amount of trainable parameters used (61 for the NN VS 15 for the PQC). The NN achieves an accuracy of 0.835 and a validation accuracy of 0.823 while the PQC again only achieves an accuracy of 0.5 by assigning the same label to the whole dataset For encoding 3 however the PQC achieves an accuracy of 0.821 on the train set and 0.8 on the test set.

So in essence -at least for the second dataset- increasing the variational circuit does not offer much but increasing the encoder complexity allows the PQC to generalize better while having comparable performance with the NN.

4.3 Dataset 3

Dataset 3 is the most complex of them all and is expected that both networks are going to have a hard time achieving a good accuracy. We expect that a little bit more expressive power is needed for the networks to do a good job so we will work in the 'deep' regime.

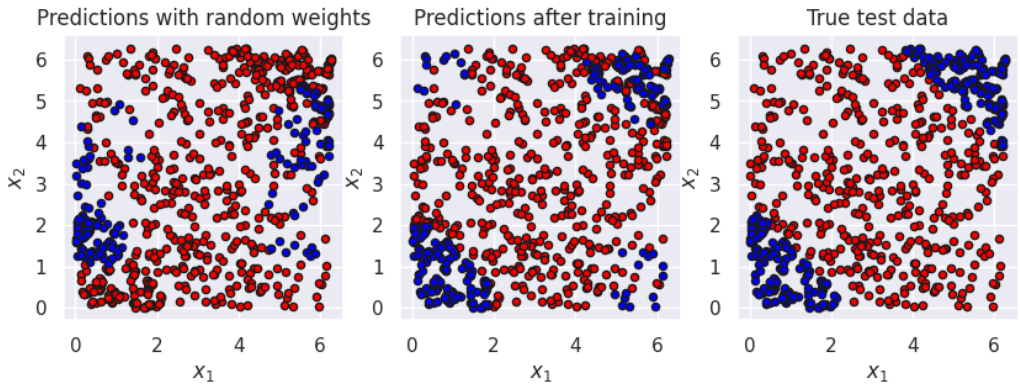


Figure 11: Caption

Now using the 'Encoding 2' we get a PQC that slightly underperforms its shallow counter part with 'Encoding 3' in its accuracy (0.808 vs 0.849)(see 9) but outperforms it in the validation set (0.812 vs 0.821). So moving from a shallow to a deep regime while moving from a 'shallower' ('Encoding 2') to a 'deeper' ('Encoding 2') encoding slightly decreases the train accuracy but increases the validation accuracy. Results for the deep PQC can be found in 11

5 Conclusions

The following table sums the result from the training and testing. The sets on the y axis reffer on the dataset used, and on the x axis is the architecture. 'S' and 'D' denote the typo of architecture (shallow or deep) and 'E1,2,3' denotes the encoding scheme used for the PQCs. The train/test accuracies are cited as presented above while values in parentheses are results omitted from the paper.

	SNN	DNN	SE1	SE2	SE3	DE1	DE2	DE3
Set 1	959/917	(998/992)	500/500	500/500	500/500			610/640
Set 2	733/718	823/835	500/490	783/789	823/769	500/499	821/800	
Set 3	578/530	(623/589)			539/544			633/593

It is observed that more complex encodings are generally better for classification (even though the encodings used are relatively simple) and increasing the variational circuit depth does not present much of an advantage. All in all none of the two architectures seems to be overall better than the other however taking into account the fact that the NN has $6 + 9l$ parameters and the PQC only $3l$ for l layers it is clear that the PQC can do 'more with less'. It is, though, evident that an encoding even more complex than 'Encoding 3' i going to be needed for a PQC to classify anything resembling real world data.

6 Further Work

The above results can only be used for ome first qualitative analysis between the two models. If one were to produce more robust results the models would need to be ran multiple times for the same parameters and the average value of the accuracies be taken into account. Here statistical anomalies due to the networks staring weights may have affected our results.

Furthermore, clearer datasets can be produced so the results are closer to accuracy 1, I tarted implementing a seperation in the dataset (which is still in the code) however at the end it did not see the light of day.

In addition, maybe the type of encodings used are not representative enough to draw robust conclusions I could have added more encoding layers but I wanted to keep the architecture as shallow as i could for computational times sake

A Part 1

A.1 Strongly Correlated Systems

A.1.1

(a) The fermionic creation and annihilation operators obey the following anti-commutation relations

$$\{a_i, a_j\} = \{a_i^\dagger, a_j^\dagger\} = 0$$
$$\{a_i, a_j^\dagger\} = \delta_{ij}$$

(b) We transform creation and annihilation operators into Majorana operators by by

$$c_{j,0} = a_j + a_j^\dagger$$
$$c_{j,1} = i(a_j - a_j^\dagger)$$

(c) The Majorana operators obey the following anti-commutation relation

$$\{c_{i,\alpha} c_{j,\beta}\} = 2\delta_{\alpha,\beta}\delta_{i,j}$$

A.1.2

A.1.3

The Hamiltonian for the Fermionic Hubbard model on a 1-dimensional chain is

$$\hat{H} = -t \sum_{\langle i,j\rangle,\sigma} \left(\hat{c}_{i,\sigma}^\dagger \hat{c}_{j,\sigma} + \hat{c}_{j,\sigma}^\dagger \hat{c}_{i,\sigma}\right) + U \sum_i \hat{n}_{i,\uparrow}\hat{n}_{i,\downarrow} + \mu \sum_{i,\sigma} \hat{n}_{i,\sigma}$$

A.1.4

A.1.5

A.2 Quantum Machine Learning

A.2.1

In the quantum variational classifier the data is encoded into a state $x \in \Omega \mapsto |\psi\rangle = |0\rangle^{\otimes n}$ and then a variational circuit $W(\theta)$ is applied to it.The final state is measured by a measurement operator M_y. Labels are assigned according to the probability of measuring the value y. An empirical risk is then used as a loss function and classical optimization methods are employed to find the parameters θ (for 0 bias) that minimize the loss.

In the quantum Kernel estimator the quantum computer estimates the kernel $K(x_i, x_j) = |\psi_i\rangle\langle\psi_j|$, $x_i, x_j \in \mathcal{S}$, where $\mathcal{S}$ is the training set and then a classical optimizer is used to maximize

$$\sum_i \alpha_i - \frac{1}{2}\sum_{i,j} y_i y_j \alpha_i \alpha_j K(x_i, x_j) \quad \text{s.t} \quad \sum_i \alpha_i y_i = 0$$

Afterwards the quantum computer is used again to estimate a new kernel $K(x_i, s)$ with $s \in \mathcal{S}$ and by using the optimal solution α^* the dataset is classified as

$$m(s) = sign\big(\sum_i y_i \alpha_i^* K(x_i, s)\big)$$

A.2.2

(a)Feature maps map the input data from its original feature space to a higher-dimensional feature space. This transformation is of the form $x \mapsto \Phi(x)$ eg $(x1, x2) \mapsto (x_1, x_2, x_1^2, x_2^2, \cdots)$ The purpose of applying a feature map is to transform the input data into a higher dimensional space where it is more easily separable by a linear classifier.

(b)Kernels are functions that measure the similarity or distance between pairs of data points in the feature space. In SVMs, kernels are used to implicitly compute the dot product between feature vectors in the transformed $K(x, y) = \Phi(x)\Phi(y)$ space without explicitly calculating the transformation. This is known as the kernel trick and avoids the computational complexity associated with explicitly working in high-dimensional spaces.

(c)A kernel has a corresponding feature map when it satisfies Mercer's condition which ensures that the kernel corresponds to an inner product in some higher-dimensional feature space. Mercer's condition states that a kernel function $K(x, y)$ is valid if and only if for any set of data points $x1, x2, \cdots, xn$, the matrix $K_{ij} = K(x_i, xj_)$ should be positive semi-definite, meaning that $z^T K z \geq 0$ for any vector y,.

A.2.3

A quantum computer is equivalent to a Turing machine so in essence anything computable (or learnable) on a classical machine can be learnt bya quantum machine. However quantum computers offer higher computational speeds on a variety of problem (e.g quantum state classification).

A.2.4

Let us suppose that the system is initially at the state

$$|\psi(0)\rangle = |0\rangle^{\otimes n}$$

and we apply to it a succesion of gates $\mathcal{U}_k$ such that the final state is

$$|\psi(t_f)\rangle = \prod_{i=1}^{d} \mathcal{U}_i |\psi(0)\rangle$$

then at any timestep $k \in [0, d]$ the probability of the first qubit being in state $|0\rangle$ is going to be

$$P_0(k) = \left| \langle 0|^{(1)} \prod_{i=1}^{k} \mathcal{U}_i |\psi(0)\rangle \right|^2$$

where $\langle 0|^{(1)}$ denotes the bra of the first qubit being in state 0 assuming we have perfect knowledge of the gates of the circuit and zero errors we can contract the product $\prod_{i=1}^{k} \mathcal{U}_i|\psi(0)\rangle$ to a simple matrix multiplication Ax which can be performed on a classical computer .